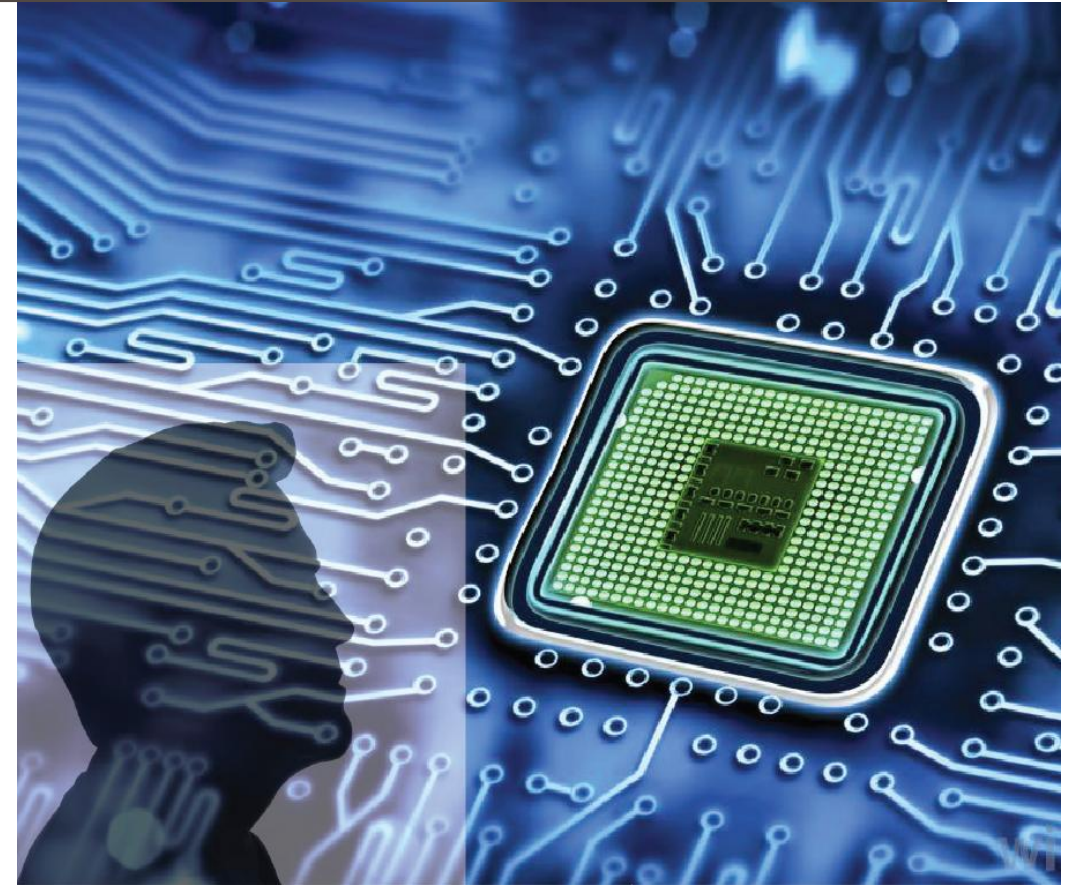


ADVANCED DIGITAL DESIGN AND VERIFICATION

Week-4 Lecture-2 Virtual Platforms

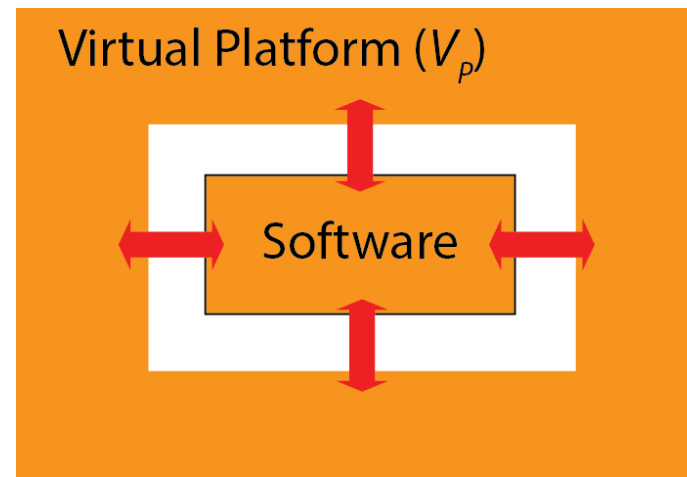
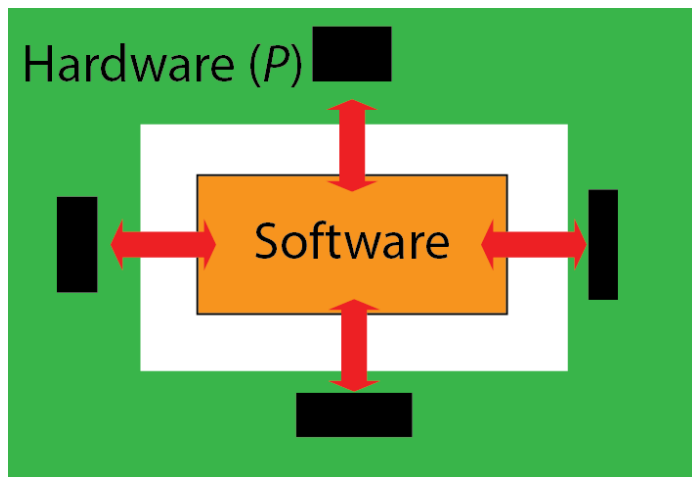
Sneh Saurabh
4th September, 2025



Virtual Platforms

Virtual Platform: Definition

- The virtual platform V_P of a hardware platform P is an abstract software model implementing the high-level functionality of P .



- Also now referred to as **Digital Twin**
- **Digital Twin**: is a virtual representation of an object or system designed to reflect a physical object accurately

- Software intended to run on P is then developed and validated on V_P

How to build:

- Typically employs TLM (Transaction Level Model) portion of the IPs

Alam, Tashfia, Indira Bhoomareddy Ramaiah, and Sandip Ray. "VirSoC: Automatic Synthesis of Virtual System-on-Chip Environments." *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2024).

Virtual Platform: Application and Benefits

Application:

- Software development:
 - Start early: applicable for entire software stack
 - Improved productivity and lower development cost
- Eases architectural exploration due to higher level of abstraction
 - Example: the pipelining of the processor or the internal implementation of the UART need not be known
- Early engagement with customers and ecosystem partners
- Allows platform implementers to share design and fabrication costs among many designers

Benefits:

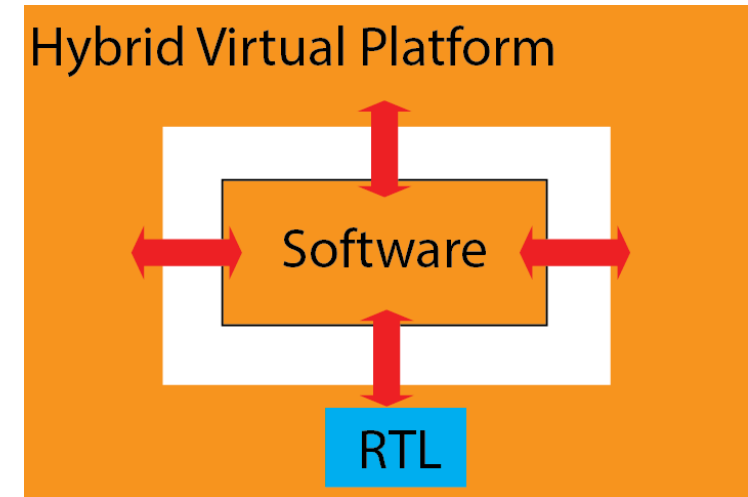
- Fast execution speed
- Scalability
- Early availability
- Easy hardware controllability, observability, determinism
- Easy distribution

Limitation:

- Corner cases of HW-SW interactions can be missed when running software on the virtual platform V_p
 - Because abstraction removes some details of the corner cases

Hybrid Virtual Platform

- Integration of some functionality as hardware models, e.g., as RTL blocks, while the rest of the functionality is in software prototypes
- Hybrid virtual platforms are popular in the industry



Examples of Virtual Platforms (1)

Tool	Company	Capabilities
------	---------	--------------

Examples of Virtual Platforms (1)

Tool	Company	Capabilities
Virtual Development Kit (VDK)	Synopsys	Provides debug and analysis tools for the software development by assembling TLMs of IPs (ARM, Synopsys DW, etc.), drivers, and OS
Helium and Hybrid Studio	Cadence	Pre-silicon software bring-up and pre-silicon HW/SW software co-verification [by integrating with Xcelium (logic simulator), Palladium (emulator) and Protium (FPGA prototyping)]
Vista Flow	Siemens EDA	Enables building TLM, architecture exploration, and software development

Examples of Virtual Platforms (2)

Tool	Company	Capabilities

Examples of Virtual Platforms (2)

Tool	Company	Capabilities
ARM Virtual Hardware	ARM	Scales and accelerates software development (before hardware is available) by providing simulation environment using cloud with TLM in the background
Simulink Coder	MathWorks	Generate SystemC/TLM components using Simulink Coder
QEMU	Open-source	Virtualization and can emulate entire systems based on x86, ARM, SPARC and other platforms along with their peripheral devices.
COSIDE	COSEDA	Integrated design environment for heterogeneous system modeling using SystemC and SystemC-AMS: focus on mixed-signal applications, such as communication, RF, automotive and sensor applications

Virtual Platforms

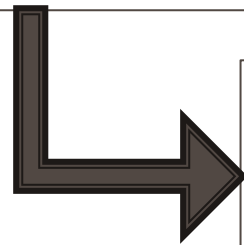
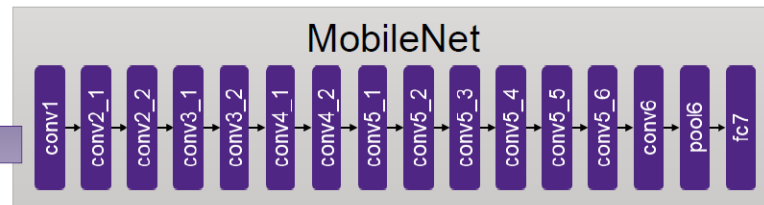
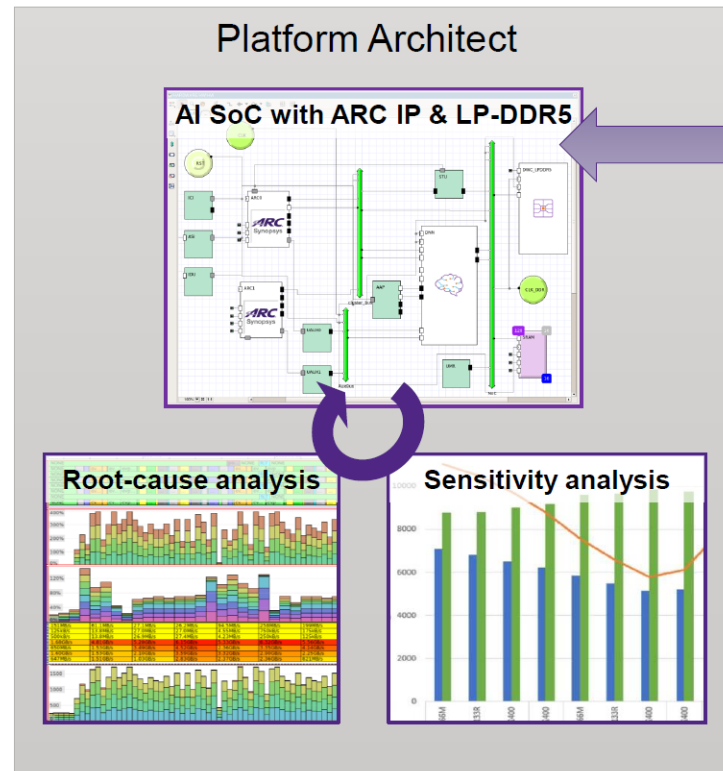


Illustration of Architectural Exploration

Problem Definition

AI SoC Platform Case-study with ARC AI Subsystem



Goals:

- ① 4 ms latency for inference of 5 frames
- ② minimize DNN power and energy

Optimize Hardware configuration:

- IP configuration
- Speed of DDR memory
- Interconnect, buffers, transactions

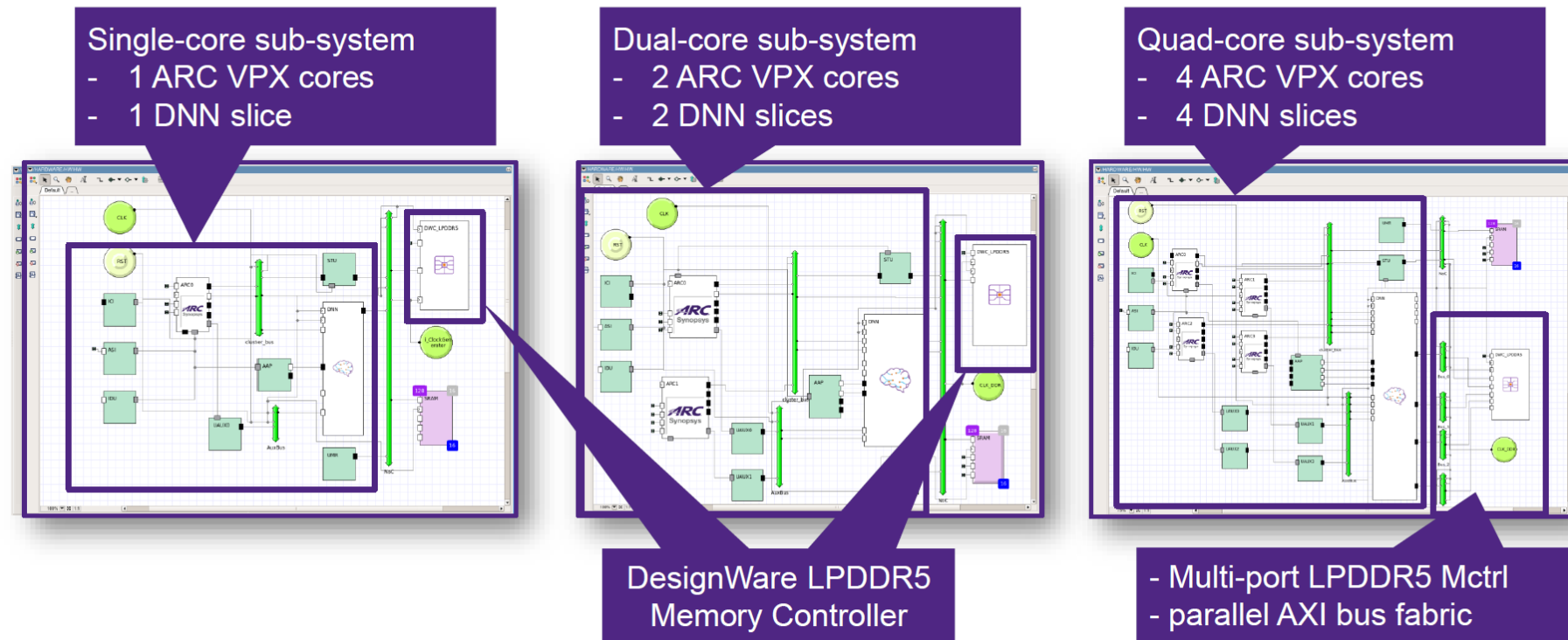
ARC:

- Argonaut RISC Core
- configurable and extensible
- extra instructions, registers, and functions

Different Options

AI SoC Block Diagram in Platform Architect

Scaling AI Sub-system and LPDDR5 memory controller



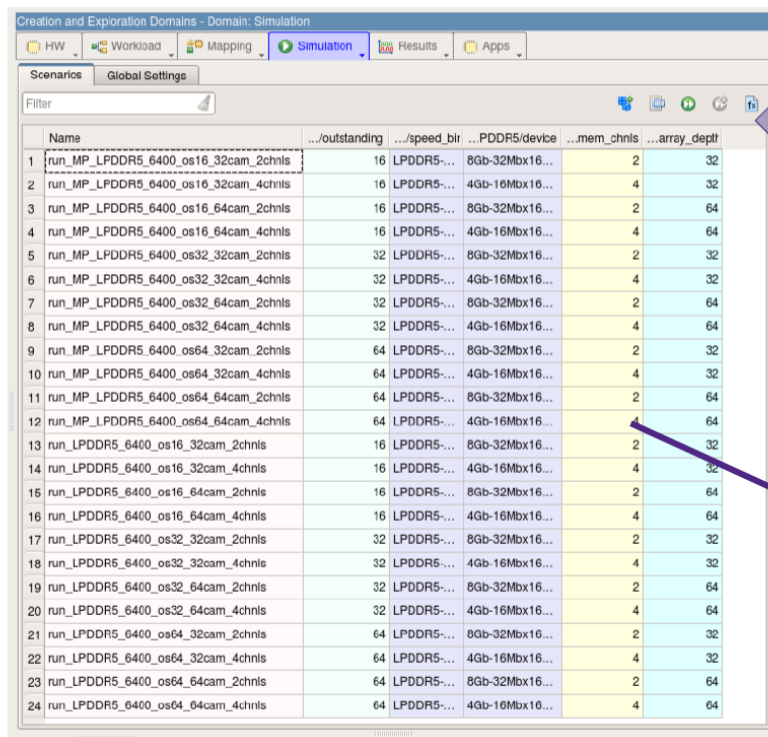
Analysis

AI SoC Architecture Sweep

Goal: 4 ms inference latency, minimize power & energy

Sweep parameters

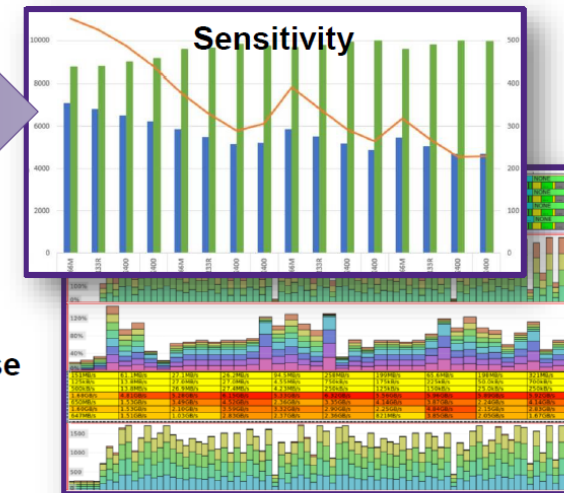
- AI configuration: 1, 2, 4 DNN slices
- Outstanding transactions: 16, 32, 64
- LPDDR5 memory speed: 3733, 4800, 6400
- Interconnect/LPDDR controller: single port, multi-port
- LPDDR controller scheduler queue: 32, 64
- LPDDR channels: 2, 4



Name	...outstanding	...speed_bir	...PDDR5/device	...mem_chnl	...array_dept
1 run_MP_LPDDR5_6400_os16_32cam_2chnls	16	LPDDR5...	8Gb-32Mbx16...	2	32
2 run_MP_LPDDR5_6400_os16_32cam_4chnls	16	LPDDR5...	4Gb-16Mbx16...	4	32
3 run_MP_LPDDR5_6400_os16_64cam_2chnls	16	LPDDR5...	8Gb-32Mbx16...	2	64
4 run_MP_LPDDR5_6400_os16_64cam_4chnls	16	LPDDR5...	4Gb-16Mbx16...	4	64
5 run_MP_LPDDR5_6400_os32_32cam_2chnls	32	LPDDR5...	8Gb-32Mbx16...	2	32
6 run_MP_LPDDR5_6400_os32_32cam_4chnls	32	LPDDR5...	4Gb-16Mbx16...	4	32
7 run_MP_LPDDR5_6400_os32_64cam_2chnls	32	LPDDR5...	8Gb-32Mbx16...	2	64
8 run_MP_LPDDR5_6400_os32_64cam_4chnls	32	LPDDR5...	4Gb-16Mbx16...	4	64
9 run_MP_LPDDR5_6400_os64_32cam_2chnls	64	LPDDR5...	8Gb-32Mbx16...	2	32
10 run_MP_LPDDR5_6400_os64_32cam_4chnls	64	LPDDR5...	4Gb-16Mbx16...	4	32
11 run_MP_LPDDR5_6400_os64_64cam_2chnls	64	LPDDR5...	8Gb-32Mbx16...	2	64
12 run_MP_LPDDR5_6400_os64_64cam_4chnls	64	LPDDR5...	4Gb-16Mbx16...	4	64
13 run_LPDDR5_6400_os16_32cam_2chnls	16	LPDDR5...	8Gb-32Mbx16...	2	32
14 run_LPDDR5_6400_os16_32cam_4chnls	16	LPDDR5...	4Gb-16Mbx16...	4	32
15 run_LPDDR5_6400_os16_64cam_2chnls	16	LPDDR5...	8Gb-32Mbx16...	2	64
16 run_LPDDR5_6400_os16_64cam_4chnls	16	LPDDR5...	4Gb-16Mbx16...	4	64
17 run_LPDDR5_6400_os32_32cam_2chnls	32	LPDDR5...	8Gb-32Mbx16...	2	32
18 run_LPDDR5_6400_os32_32cam_4chnls	32	LPDDR5...	4Gb-16Mbx16...	4	32
19 run_LPDDR5_6400_os32_64cam_2chnls	32	LPDDR5...	8Gb-32Mbx16...	2	64
20 run_LPDDR5_6400_os32_64cam_4chnls	32	LPDDR5...	4Gb-16Mbx16...	4	64
21 run_LPDDR5_6400_os64_32cam_2chnls	64	LPDDR5...	8Gb-32Mbx16...	2	32
22 run_LPDDR5_6400_os64_32cam_4chnls	64	LPDDR5...	4Gb-16Mbx16...	4	32
23 run_LPDDR5_6400_os64_64cam_2chnls	64	LPDDR5...	8Gb-32Mbx16...	2	64
24 run_LPDDR5_6400_os64_64cam_4chnls	64	LPDDR5...	4Gb-16Mbx16...	4	64



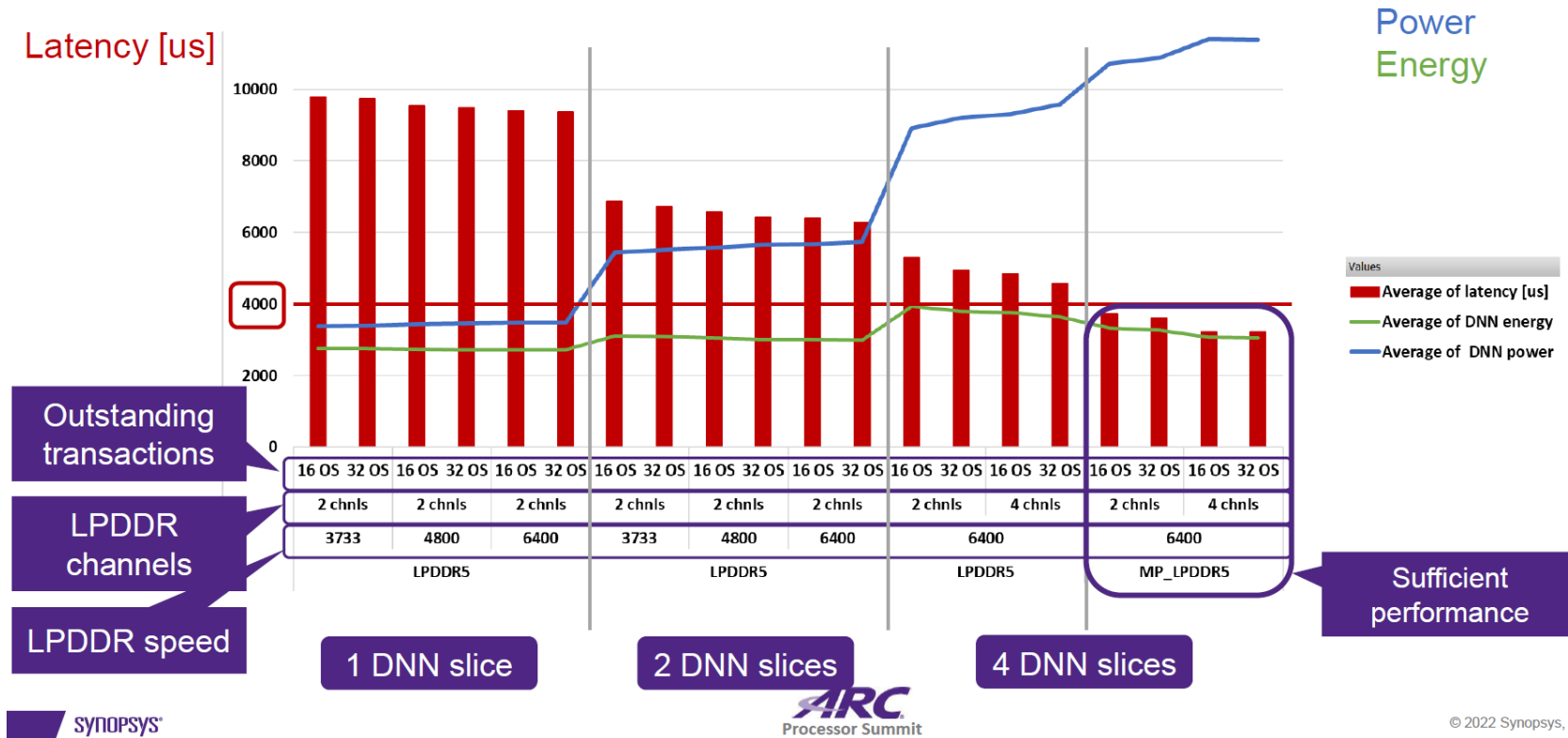
Root-Cause
Analysis



Optimization

Analysis and Optimization of Architecture Configurations

Inference latency for 5 frames vs. DNN power and energy consumption

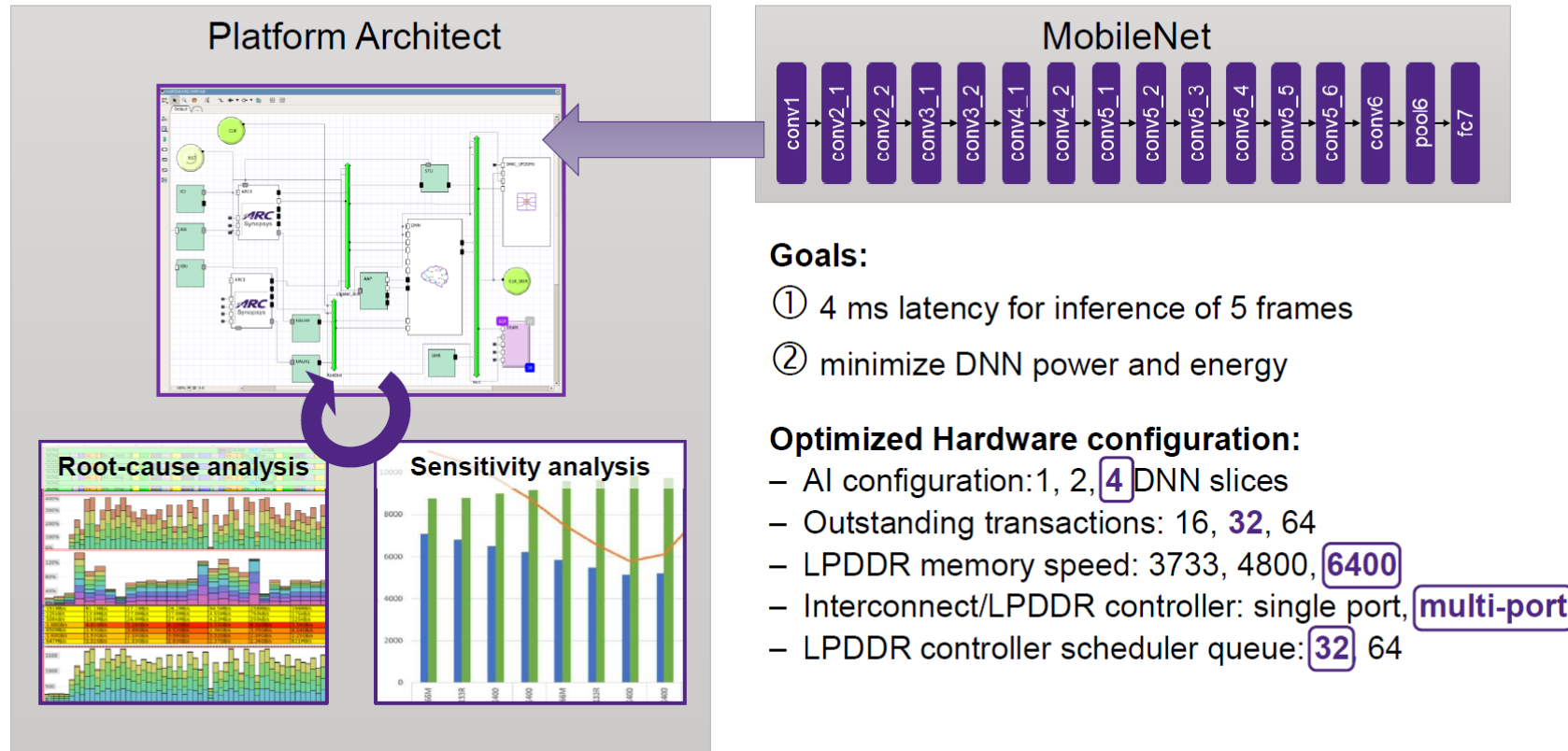


<https://www.synopsys.com/verification/virtual-prototyping/platform-architect.html>

Advanced Digital Design and Verification S. Saurabh

Final Result

Example Summary



<https://www.synopsys.com/verification/virtual-prototyping/platform-architect.html>

Partitioning

Partitioning: Basics

Definition:

- Process of subdividing an initial specification into a set of processes (possibly concurrent) and assigning them to:
 - software running on processors,
 - silicon area or IP blocks for hardware,
 - communication channels (e.g., buses)
 - storage resources (e.g., memories).

Costs: PPA, QoS, fault tolerance etc.

Constraints: worst-case execution time, response time, cost, component reuse, design team experience, memory size for software, etc.

Examples:

- Associating an FFT or a filter with a CPU or DSP or piece of dedicated hardware
- Associating an abstract FIFO channel or an array of integers with a driver, a bus and some interfaces, a segment of a RAM
 - Size of the FIFO channel, priority among tasks

Partitioning: Merits and Demerits

Merits

- Exploit the different capabilities of hardware and software
 - Satisfy different flexibility, performance, and power requirements.
- Speed up the overall design process
 - Allowing teams to work in relative independence
 - Need to obey the required interfacing constraints and satisfy their local functional specifications.
- Simplify the verification of each partition in isolation.

Demerit

- Reduces optimization opportunities by introducing barriers
 - Very expensive or impossible to cross boundaries

Partitioning: Key Aspects

Key aspects:

- Functional Decomposition
- Architecture Description
- Partitioning approaches
- Hardware partition, Software Partition, Communication

Partitioning: Functional Decomposition (1)

Functional decomposition

- Specification should expose substantial application-level parallelism for mapping

Use Concurrent Language for a domain

- Designer must explicitly expose all the concurrency that will be exploited by partitioning and mapping.
- Example: Simulink (MathWorks), LabView, Lustre (Estrel)
- Simulink allows decompose a control or signal processing algorithm into a block-level diagram
 - Each block is potentially concurrent with all others, except for data dependencies and execution rate constraints.

Use sequential language + Tool to extract concurrency

- Specification in C, C++
 - Tools extract specification parallelism automatically
 - Challenging problem, and concurrent language preferable
-
- Typically loops and arrays are target for parallelization
 - Loops often consume most computational and storage resources
 - Arrays reading/writing often is the communication bottleneck

Partitioning: Functional Decomposition (2)

```
for (int i=0; i<100; i++)  
    a[i] = b[i] + c;
```

- There are no data dependencies between iterations
- Can be arbitrarily parallelized

```
for (int i=0; i<100; i++)  
    a[i+2] = a[i] + c;
```

- Involve dependencies between successive iterations
- Distance between dependent executions of the loop body can be statically analyzed.

```
for (int i=0; i<100; i+=2)  
    a[i+2] = a[i] + c;
```

```
for (int i=1; i<100; i+=2)  
    a[i+2] = a[i] + c;
```

- Odd and even iterations never interact
- Two concurrent programs can be immediately extracted

- Which one will be better?
- It will depend on the architecture

- Dual core with less parallelism: two processes may be better
- Superscalar (instruction level parallelism within CPU, multiple instructions within the same clock cycle): one process may be better

Partitioning: Architecture Description

- Architecture description: collection of components that provide a means to implement a behavior

Three broad classes:

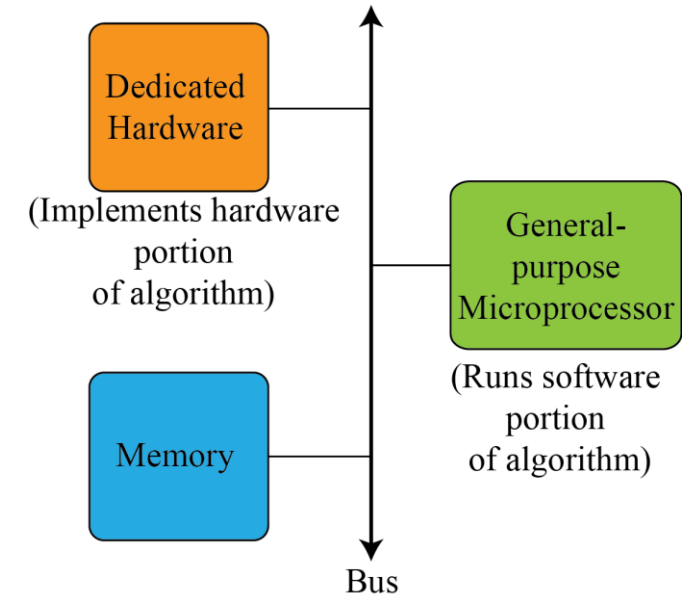
- Computation
- Communication
- Storage

Simple Architecture:

- CPU, Accelerator, FPGA
- Bus
- Memory

Other components: network protocol, instruction set, arbitration policies

Virtual platform: can be considered as architecture description which is re-usable



Partitioning problem boils down to assigning each functional module to an architectural component such that:

1. Functionality is met
2. Performance (FoM) goals are met

Architecture Components (1)

CPU

- Key parameters of CPU
 - Clock frequency
 - Instruction set
 - Power consumption
 - Number of pipeline stages
 - Instruction issue parallelism
 - Embedded memories and caches
- Can be at various levels of abstraction
 - Instruction set models: cycle accurate ISS, functional instruction-accurate
 - RTL

Configurable CPUs

- ISA and their communication interfaces (memory access channels, FIFOs, etc.) can be extended to fit the application at hand
- If this is sufficient to meet PPA targets, then custom hardware is avoided
- Tools that generate a particular instance of the configurable/extensible processor can generate:
 - Cycle-accurate ISS model
 - Fast-functional ISS model.
 - Compiler, assembler, and integrated software development environment

Architecture Components (2)

Custom hardware

- Components are generally developed for a particular application in an ad hoc manner
- RTL often available
 - An abstracted view can describe architecture
 - Key attributes can be used in architecture description: maximum clock frequency, maximum power dissipation, available area, and number of pins

Standard Hardware

- Peripherals and accelerators
- Used in many systems
 - Accurate models are built: RTL or behavioral

Architecture Components (3)

Buses

- Communication often modeled as TLM
- Main parameters:
 - Width of addresses and data,
 - Kind of transactions: read, write, burst, split
 - Pipelining options
 - Number of masters and slaves
 - Arbitration policies
 - Maximum clock frequency for synchronous buses

Memory

- Type and size
- Number of address and data lines
- Number of read and write ports
- Access time

Operating system

- Essential role is to schedule several concurrent tasks on the same CPU and to manage the shared hardware resources
- Scheduling policies: effects of concurrency and pre-emption
- At architecture description level: reaction time and completion time needs to be modeled

References

- Martin, Grant, Brian Bailey, and Andrew Piziali, “ESL design and verification: a prescription for electronic system level methodology”, Elsevier, 2010.